

Lekcja 13 - Spark Internals

2026-09

Data Engineering Course

Spark Internals

Jak Spark naprawdę wykonuje job?
Shuffle, skew, broadcast join i partiowania

■ Problem biznesowy

Job działa, ale działa coraz wolniej:

5 min -> 40 min -> 3 h

To nie jest przypadek. To jest sygnał, że problem nie leży w SQL, tylko w planie wykonania.

W praktyce: "to działa" nie oznacza "to jest skalowalne".

■ Co naprawdę trzeba zrozumieć

Spark nie wykonuje jednego wielkiego kroku.
Dzieli pracę na stages, tasks i partitions.

Najczęstszy błąd początkujących: kod jest poprawny, ale operacja robi zbyt dużo danych przez sieć i pamięć.

■ First principles

Spark nie wykonuje jednego wielkiego kroku.
Dzieli pracę na stages, tasks i partitions.

W praktyce koszt operacji rośnie tam, gdzie pojawia się shuffle i nierównomierny rozkład danych.

■ Agenda

1. DAG i job
2. Stage / task / partition
3. Shuffle i skew
4. Joins i broadcast join
5. Partitioning, repartition, coalesce
6. Homework + interview

```
read -> filter -> join -> aggregate -> write
```

Spark buduje plan jako DAG. Każda operacja ma zależności. Nie pracuje „na bieżąco”, tylko planuje wykonanie.

■ Stage i task

Stage = część joba między shuffle
Task = obliczenie na jednej partycji

Dużo małych zadań = dobre rozproszenie. Za mało partycji = słabe wykorzystanie zasobów.

■ Shuffle

shuffle = redistribute data across executors

To jest najdroższa operacja w Spark. W praktyce pojawia się przy:

- `groupBy`
- `join`
- `distinct`
- `sort`

Jedna partycja ma 10x więcej danych niż reszta.

Skew powoduje, że 1 executor pracuje dużo dłużej niż pozostali. To jest klasyczny bottleneck w dużych batchach.

■ Broadcast join

```
from pyspark.sql.functions import broadcast  
  
df_large.join(broadcast(df_small), "customer_id")
```

Gdy jedna tabela jest mała, wysyłasz ją do wszystkich executorów.
To eliminuje drogi shuffle.

■ Join decision tree

```
mała tabela -> broadcast join  
duże tabele -> sort merge join / shuffle join
```

Wybór joinu to decyzja o koszcie i skalowalności, nie tylko stylu kodu.

■ Partitioning

```
repartition = zwiększa/zmienia liczbę partycji  
coalesce = zmniejsza liczbę partycji bez shuffle
```

Błędy w partycjonowaniu są często przyczyną wolnych pipeline'ów.

■ Practical checklist

1. Czy widzisz dużo shuffle w EXPLAIN?
2. Czy jedna partycja dominuje rozmiarem?
3. Czy masz join między dużymi tabelami?
4. Czy zapis jest zbyt mało / zbyt dużo partycjonowany?

Q: Why is shuffle expensive?

A: Because it moves data over the network and forces stages to synchronize across executors.

■ Homework

Przeanalizuj job, znajdź 2 bottlenecks,
zoptymalizuj join i przedstaw wynik w 5-8
zdaniach.

